MUSTAFACODE

Lecture 17

Object Oriented Programming (CS304)
Object-Oriented Programming (CS304)

Course Code: **CS304**

Publisher: **mustafacode**

Compiled On: **May 29, 2026**

CS304 OBJECT-ORIENTED PROGRAMMING

Lecture 17: Object Oriented Programming (CS304)

Object Oriented Programming (CS304)

Lecture 17 — Binary Operator Overloading & Assignment Operator

---

Introduction

Operator Overloading ka matlab hai existing operators (`+` , `-` , `*` , `=` etc.) ko apni class ke objects ke liye customize karna.

C++ humein allow karta hai ke hum operators ko apni requirement ke mutabiq use kar saken.

Binary Operators

Binary operators wo operators hote hain jo 2 operands ke sath kaam karte hain.

Examples

```
a + b
a - b
a * b
a / b
a = b
```

Yahan:

- `a` → left-hand operand
- `b` → right-hand operand

Important Rule

Binary operator hamesha left-hand object ke reference se call hota hai.

Example

```
c1 + c2
```

Internally compiler isko aise call karta hai:

```
c1.operator+(c2)
```

Yahan:

- `c1` operator ko call kar raha hai
- `c2` argument ke taur par pass ho raha hai

Another Example

```
c2 + c1
```

Internally:

```
c2.operator+(c1)
```

Complex Class Example

Step 1 — Basic Complex Class

```cpp
#include <iostream>
using namespace std;

class Complex {

private:
    double real;
    double imag;

public:

    Complex(double r = 0, double i = 0) {
        real = r;
        imag = i;
    }

    void display() {
        cout << real << " + " << imag << "i" << endl;
    }

    Complex operator+(const Complex& rhs) {

        Complex temp;

        temp.real = real + rhs.real;
        temp.imag = imag + rhs.imag;

        return temp;
    }
};

int main() {

    Complex c1(2,3);
    Complex c2(4,5);

    Complex c3 = c1 + c2;

    c3.display();

    return 0;
}
```

Output

```
6 + 8i
```

Real-Life Example

Socho:

- `real` = mango juice
- `imag` = orange juice

Jab hum:

```
c1 + c2
```

karte hain to dono juices mix ho jate hain.

Problem

Ye code sirf:

```
Complex + Complex
```

handle karta hai.

Lekin:

```
c1 + 5.5
```

handle nahi karega.

Solution — Overload for double

Modified Class

```cpp
#include <iostream>
using namespace std;

class Complex {

private:
    double real;
    double imag;

public:

    Complex(double r = 0, double i = 0) {
        real = r;
        imag = i;
    }

    void display() {
        cout << real << " + " << imag << "i" << endl;
    }

    // Complex + Complex
    Complex operator+(const Complex& rhs) {

        Complex temp;

        temp.real = real + rhs.real;
        temp.imag = imag + rhs.imag;

        return temp;
    }

    // Complex + double
    Complex operator+(const double& rhs) {

        Complex temp;

        temp.real = real + rhs;
        temp.imag = imag;

        return temp;
    }
};

int main() {
```

```
    Complex c1(5,2);

    Complex c2 = c1 + 10.5;

    c2.display();

    return 0;
}
```

Output

```
15.5 + 2i
```

Problem Again

Ye abhi bhi handle nahi karega:

```
10.5 + c1
```

Why Error?

Compiler internally likhta hai:

```
10.5.operator+(c1)
```

Lekin basic data type `double` ke paas overloaded operator nahi hota.

Friend Function Solution

Hum non-member friend function use karenge.

Friend Function Example

```
#include <iostream>
using namespace std;

class Complex {

private:
    double real;
    double imag;

public:

    Complex(double r = 0, double i = 0) {
        real = r;
        imag = i;
    }

    void display() {
        cout << real << " + " << imag << "i" << endl;
    }

    friend Complex operator+(const Complex& lhs, const double& rhs);

    friend Complex operator+(const double& lhs, const Complex& rhs);
};

Complex operator+(const Complex& lhs, const double& rhs) {

    Complex temp;

    temp.real = lhs.real + rhs;
    temp.imag = lhs.imag;

    return temp;
}

Complex operator+(const double& lhs, const Complex& rhs) {

    Complex temp;

    temp.real = lhs + rhs.real;
    temp.imag = rhs.imag;

    return temp;
}
```

```
int main() {

    Complex c1(4,3);

    Complex c2 = 5.5 + c1;

    c2.display();

    return 0;
}
```

Output

```
9.5 + 3i
```

Other Binary Operators

Bilkul isi tarah hum:

```
*
/
-
```

operators bhi overload kar sakte hain.

Example

```
Complex operator*(const Complex&, const Complex&);

Complex operator/(const Complex&, const Complex&);

Complex operator-(const Complex&, const Complex&);
```

Assignment Operator Overloading

Compiler Automatically Generates

Agar hum khud na likhen to compiler:

1. Default Constructor
2. Copy Constructor
3. Assignment Operator

khud generate karta hai.

Problem with Dynamic Memory

Agar class dynamic memory use kare:

```
new
```

to compiler shallow copy karta hai.

Isse:

- memory leak
- dangling pointer

jaise issues aate hain.

String Class Example

```
class String {

private:
    int size;
    char* bufferPtr;

public:

    String() {
        bufferPtr = NULL;
        size = 0;
    }
};
```

Parameterized Constructor

```
String(char* ptr) {

    if(ptr != NULL) {

        size = strlen(ptr);

        bufferPtr = new char[size + 1];

        strcpy(bufferPtr, ptr);
    }
    else {

        bufferPtr = NULL;
        size = 0;
    }
}
```

Copy Constructor

```
String(const String& rhs) {

    size = rhs.size;

    if(rhs.size != 0) {

        bufferPtr = new char[size + 1];

        strcpy(bufferPtr, rhs.bufferPtr);
    }
    else {

        bufferPtr = NULL;
    }
}
```

Shallow Copy Problem

```
String str1("Hello");
String str2("World");

str1 = str2;
```

Compiler sirf address copy karta hai.

Result:

- dono objects same memory ko point karte hain
- delete hone par issues aate hain

Deep Copy Assignment Operator

Code

```
#include <iostream>
#include <cstring>

using namespace std;

class String {

private:
    int size;
    char* bufferPtr;

public:

    String(char* ptr = NULL) {

        if(ptr != NULL) {

            size = strlen(ptr);

            bufferPtr = new char[size + 1];

            strcpy(bufferPtr, ptr);
        }
        else {

            size = 0;
            bufferPtr = NULL;
        }
    }

    void display() {
        cout << bufferPtr << endl;
    }

    String& operator=(const String& rhs) {

        if(this == &rhs)
            return *this;

        delete[] bufferPtr;

        size = rhs.size;

        if(rhs.size != 0) {
```

```
            bufferPtr = new char[size + 1];

            strcpy(bufferPtr, rhs.bufferPtr);
        }
        else {

            bufferPtr = NULL;
        }

        return *this;
    }

    ~String() {
        delete[] bufferPtr;
    }
};

int main() {

    String str1("ABC");
    String str2("XYZ");

    str1 = str2;

    str1.display();

    return 0;
}
```

Output

```
XYZ
```

Why Return `String&` ?

Taake chaining possible ho:

```
str1 = str2 = str3;
```

Internally

```
str1.operator=(str2.operator=(str3))
```

`return *this`

`this` current object ko point karta hai.

```
return *this;
```

current object return karta hai.

Real-Life Example

Socho:

3 notebooks hain:

```
str1
str2
str3
```

Shallow Copy

Sirf notebook ka address share kiya gaya.

Agar ek notebook destroy hui to sab problem mein ajayenge.

Deep Copy

Har notebook mein proper notes alag copy hue.

Sab independent rahenge.

Important Concepts Summary

Concept	Explanation
Binary Operator	2 operands ke sath kaam karta hai
Operator Overloading	Operators ko customize karna
Left-hand Object	Operator hamesha left object se call hota hai
Friend Function	Non-member function jo private members access kare
Shallow Copy	Sirf address copy hota hai
Deep Copy	Actual data copy hota hai
Assignment Operator	`=` operator overload karna
`this` Pointer	Current object ko point karta hai
`return *this`	Current object return karta hai

Final Important Points

Member Function

```
c1 + c2
```

Internally:

```
c1.operator+(c2)
```

Friend Function

Useful when:

```
5 + c1
```

Deep Copy is Necessary

Agar class dynamic memory use kare to:

- Copy Constructor
- Assignment Operator
- Destructor

properly likhna zaroori hai.

End of Notes